



**Universidad Nacional
Autónoma de México**



Facultad de Ingeniería

**Laboratorio de Arquitectura y Organización
de Computadoras**

Grupo: 07 - Semestre: 2027-1

Reporte Practica 2 Pt.2

Diseño de máquinas de estado

Fecha de Entrega

18 de Septiembre del 2026

Profesor

Ing. Julio Cesar Cruz Estrada

Integrantes:

Espinoza Matamoros Percival Ulises

Montiel Juárez Oscar Iván

Veraza García Amy Valentina

1. Objetivo

Implementar en VHDL una máquina de estados que controle un contador de cuatro bits en una FPGA, sustituyendo el avance automático por un pulso de reloj generado mediante un botón. Verificar el diseño mediante simulación funcional e identificar la relación entre la carta ASM, el código VHDL y el diagrama de bloques sintetizado en Quartus.

2. Desarrollo

En esta segunda parte se conservó el comportamiento general de la práctica anterior: una máquina de estados controla el reinicio, el conteo y la señal de término de un contador de cuatro bits. La diferencia principal consiste en que la lógica de control se describió con VHDL y el reloj de avance ya no se aplica de manera automática. Cada pulsación del botón genera un único evento de reloj útil, por lo que el contador puede observarse paso a paso en la tarjeta FPGA.

El diseño se dividió en dos módulos. El módulo `clk_button` recibe el reloj de la tarjeta y la señal del botón; espera la pulsación, cuenta 25 millones de ciclos del reloj base y produce la señal `div_clk`. Este intervalo evita que los rebotes mecánicos del botón se traduzcan en múltiples avances. El módulo `state_machine` implementa los estados ED0, ED1 y ED2, y produce las salidas `reset`, `done` y `edo_p`.

2.1. Carta ASM del generador de pulso manual

La Figura 1 representa la lógica del módulo que transforma una pulsación en un flanco de reloj. La decisión `Button` determina si existe una solicitud de avance. Mientras no se presione el botón, el sistema permanece en espera. Al detectarlo, se habilita `div_clk` y se incrementa el contador interno. La decisión `max` comprueba que se hayan alcanzado los 25 millones de ciclos; al cumplirse, se termina el pulso y el sistema vuelve al estado de espera. Así, una pulsación produce un solo flanco ascendente de `div_clk` para la máquina principal.

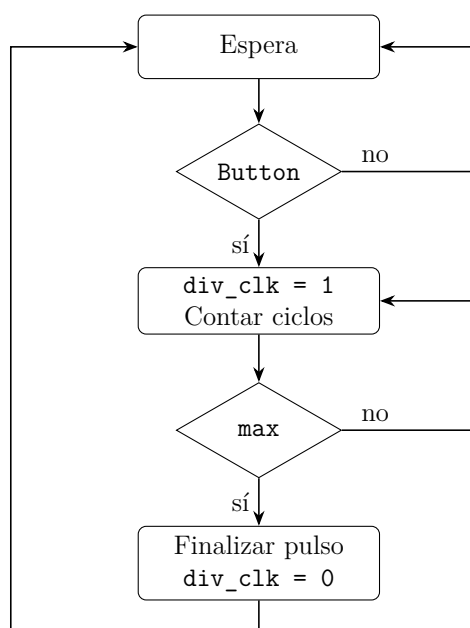


Figura 1: Carta ASM del generador de pulso manual. Las decisiones son **Button** y **max**; **div_clk** es la señal habilitada durante el conteo interno.

2.2. Descripción VHDL

La descripción VHDL permite expresar directamente los estados, las transiciones y las salidas sin recurrir a mapas de Karnaugh. Cada archivo se divide en una *entity*, que declara la interfaz del bloque, y una *architecture*, que define su comportamiento. Los tipos `std_logic` y `std_logic_vector` representan, respectivamente, una señal binaria y un conjunto ordenado de bits. La biblioteca `IEEE.STD_LOGIC_1164` proporciona dichos tipos y las operaciones básicas empleadas por los dos módulos.

En el Listado 1, la entidad `clk_button` recibe `clk`, el reloj de alta frecuencia de la tarjeta, y `button`, la señal del botón ya invertida en el diagrama de bloques. La salida `div_clk` es el reloj lento o pulso válido que se utiliza para avanzar el circuito principal. El nombre `button` se conserva tal como fue declarado en el archivo VHDL, aunque semánticamente representa al botón.

Las constantes `STATE_START`, `STATE_COUNT` y `STATE_DONE` codifican los tres estados del generador de pulso como 00, 01 y 10. La señal `state` almacena el estado actual. Por otra parte, `cuenta` es un vector de 28 bits que funciona como contador interno; `MAX_CLK_COUNT = X'17D7840'` equivale a 25 millones en hexadecimal. Al emplear un valor grande se introduce un retardo suficiente para que una pulsación física no origine varios avances debido al rebote mecánico.

El primer proceso es secuencial, pues sólo modifica `state` en el flanco ascendente de `clk`,

indicado por `rising_edge(clk)`. El bloque `case` describe las transiciones: en `STATE_START` se espera que `button = '1'`; al ocurrir, se pasa a `STATE_COUNT`. En este último estado se verifica continuamente si `cuenta` alcanzó el límite. Cuando esto sucede, se pasa a `STATE_DONE`; en el siguiente flanco se regresa a `STATE_START`. La cláusula `when others` asigna un estado seguro si ocurriera una codificación no prevista.

El segundo proceso incrementa `cuenta` únicamente cuando `state = STATE_COUNT`; fuera de ese estado lo reinicia a cero. Finalmente, el tercer proceso es combinacional: asigna `div_clk = '1'` mientras se está contando y `div_clk = '0'` en los demás estados. Por ello, el cambio de nivel de `div_clk` queda asociado a una única pulsación aceptada y proporciona el flanco que dispara la máquina principal.

El Listado 2 contiene la máquina que controla el contador de cuatro bits. Sus entradas son `clk`, `go` y `fc`; sus salidas son `reset`, `done` y `edo_p`. La señal `fc` es generada por la compuerta AND de cuatro entradas cuando el contador vale 1111. El vector `edo_p` permite observar desde fuera el estado presente y facilita la verificación en simulación.

En ED0 se desactivan `reset` y `done`; si `go` se encuentra en alto, el siguiente estado es ED1. En ED1, `reset` se activa y se mantiene hasta que aparece `fc`; entonces se selecciona ED2. En ED2, `done` toma el valor uno durante un ciclo de reloj y la máquina vuelve a ED0. De este modo, cada pulso generado a partir del botón gobierna una transición sincronizada y el contador sólo avanza bajo el control definido por la máquina de estados.

Listing 1: Módulo VHDL que genera el pulso de reloj a partir del botón.

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_ARITH.ALL;
4  use IEEE.STD_LOGIC_UNSIGNED.ALL;
5
6  entity clk_button is
7      port(
8          clk : in std_logic;
9          button : in std_logic;
10         div_clk : out std_logic
11     );
12 end clk_button;
13
14 architecture Behavioral of clk_button is
15
16     CONSTANT STATE_START: std_logic_vector(1 downto 0) := "00";
17     CONSTANT STATE_COUNT: std_logic_vector(1 downto 0) := "01";
18     CONSTANT STATE_DONE: std_logic_vector(1 downto 0) := "10";
19
20     CONSTANT MAX_CLK_COUNT: std_logic_vector(27 downto 0) := X"17D7840"; -- 25
        millones de pulsos
21
22     signal state: std_logic_vector(1 downto 0) := "00";
23     signal cuenta: std_logic_vector(27 downto 0) := X"0000000";

```

```
24
25 begin
26     process(clk, button)
27     begin
28         if rising_edge(clk) then
29             case state is
30                 WHEN STATE_START =>
31                     if button = '1' then
32                         state <= STATE_COUNT;
33                     end if;
34                 WHEN STATE_COUNT =>
35                     if cuenta >= MAX_CLK_COUNT then
36                         state <= STATE_DONE;
37                     end if;
38                 WHEN STATE_DONE =>
39                     state <= STATE_START;
40                 WHEN others =>
41                     state <= STATE_START;
42             end case;
43         end if;
44     end process;
45
46     process(clk)
47     begin
48         if rising_edge(clk) then
49             if state = STATE_COUNT then
50                 cuenta <= cuenta + 1;
51             else
52                 cuenta <= X"00000000";
53             end if;
54         end if;
55     end process;
56
57     process(state)
58     begin
59         if state = STATE_COUNT then
60             div_clk <= '1';
61         else
62             div_clk <= '0';
63         end if;
64     end process;
65
66 end Behavioral;
```

Listing 2: Máquina de estados principal en VHDL.

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3 use IEEE.STD_LOGIC_ARITH.ALL;
4 use IEEE.STD_LOGIC_UNSIGNED.ALL;
5
```

```
6 entity state_machine is
7     port(
8         clk : in std_logic;
9         go  : in std_logic;
10        fc : in std_logic;
11        reset : out std_logic;
12        done  : out std_logic;
13        edo_p : out std_logic_vector(1 downto 0)
14    );
15 end state_machine;
16
17 architecture Behavioral of state_machine is
18
19     CONSTANT ED0: std_logic_vector(1 downto 0) := "00";
20     CONSTANT ED1: std_logic_vector(1 downto 0) := "01";
21     CONSTANT ED2: std_logic_vector(1 downto 0) := "10";
22
23     signal state: std_logic_vector(1 downto 0) := "00";
24
25 begin
26     process(clk, go, fc)
27     begin
28         if rising_edge(clk) then
29             case state is
30                 WHEN ED0 =>
31                     edo_p <= ED0;
32                     reset <= '0';
33                     done  <= '0';
34                     if go = '1' then
35                         state <= ED1;
36                     end if;
37                 WHEN ED1 =>
38                     edo_p <= ED1;
39                     reset <= '1';
40                     done  <= '0';
41                     if fc = '1' then
42                         state <= ED2;
43                     end if;
44                 WHEN ED2 =>
45                     edo_p <= ED2;
46                     reset <= '0';
47                     done  <= '1';
48                     state <= ED0;
49                 WHEN others =>
50                     edo_p <= ED0;
51                     reset <= '0';
52                     done  <= '0';
53                     state <= ED0;
54             end case;
55         end if;
```

```

56     end process;
57 end Behavioral;

```

3. Simulaciones

La verificación se realizó en el *Simulation Waveform Editor* de Quartus. Se observaron las salidas del contador A, B, C y D, la bandera **done**, el vector **edo_presente**, y las entradas **go**, **button** y **clk**. En simulación, el reloj base permanece periódico, pero el avance del sistema sólo ocurre cuando el módulo del botón genera el pulso **div_clk**.

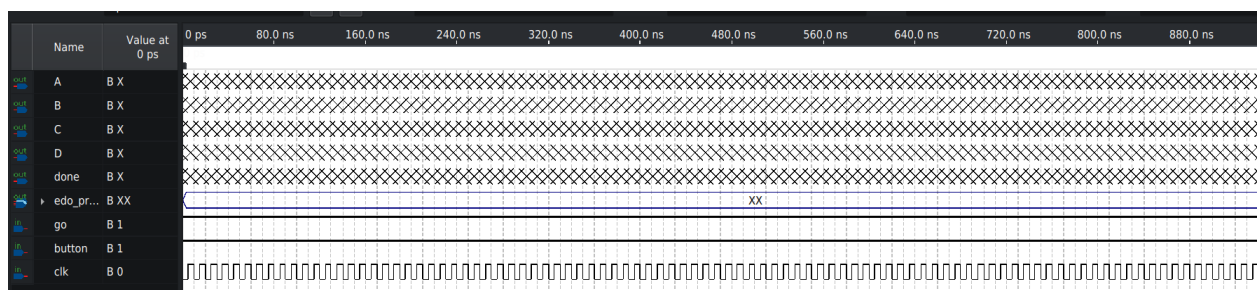


Figura 2: Condición inicial de la simulación. Las salidas sin una transición previa de reloj aparecen como valores indefinidos.

En la Figura 2, las señales de salida aparecen como X antes de que la máquina haya propagado su primer estado válido. Este resultado es normal para las salidas registradas que todavía no han sido actualizadas por un flanco efectivo. La presencia de **go**, **button** y **clk** permite establecer las condiciones que activarán el circuito durante la simulación.

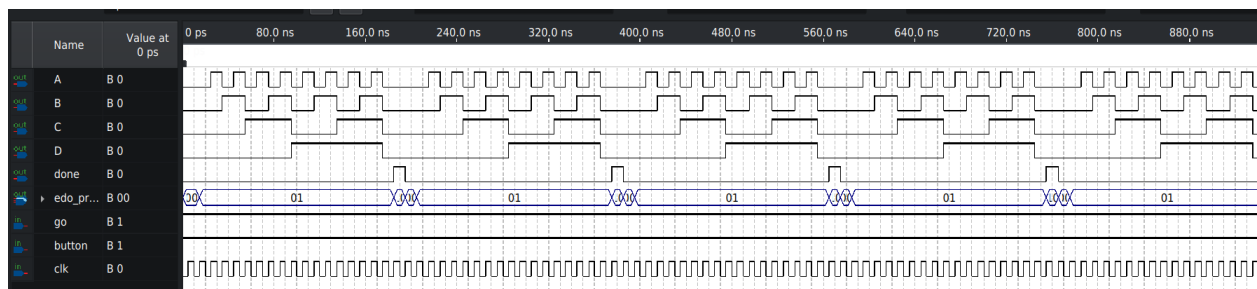


Figura 3: Evolución de las salidas del contador y del estado presente después de aplicar pulsos válidos.

La Figura 3 muestra que las salidas se estabilizan en valores binarios y que cada bit cambia a una frecuencia relativa distinta. A es el bit menos significativo y presenta la mayor tasa de cambio; B, C y D cambian cada dos, cuatro y ocho incrementos, respectivamente. El vector de

estado se mantiene en 01 durante el conteo, lo que corresponde al estado ED1 de la máquina principal.

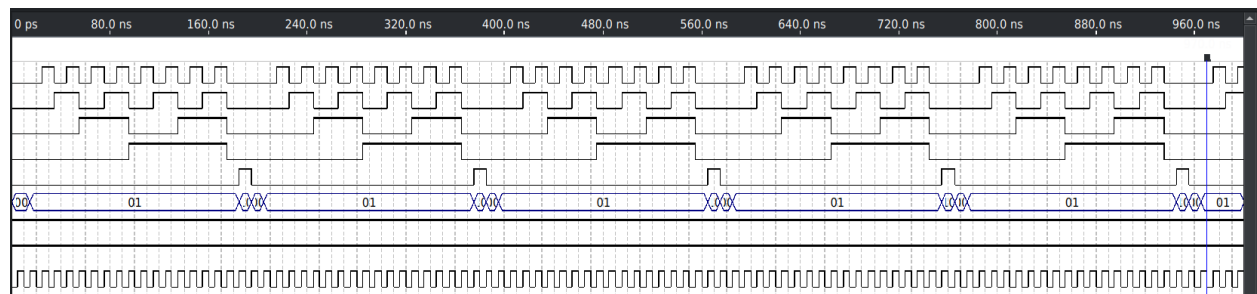


Figura 4: Simulación completa: conteo de cuatro bits, estado presente y pulsos de término.

En la Figura 4 se reconoce la secuencia de conteo y la activación puntual de **done**. Al alcanzar la combinación máxima, la detección de cuenta final **fc** provoca la transición hacia ED2; la salida **done** se activa durante un ciclo y la máquina vuelve al estado de espera. El uso del generador de pulso manual hace que estas transiciones dependan de la pulsación del botón y no de un reloj automático visible para el usuario.

4. Resultados

La Figura 5 muestra el sistema integrado en Quartus. El reloj de la tarjeta entra por el pin P11 y el botón por B8; la señal del botón se invierte antes de ingresar al módulo `clk_button`, pues en la tarjeta se utiliza lógica activa en bajo. La salida `div_clk` alimenta a la máquina de estados y al contador 74193. La entrada `go` se asigna al pin C10, `done` al pin B11 y las cuatro salidas del contador se conectan a A8, A9, A10 y B10.

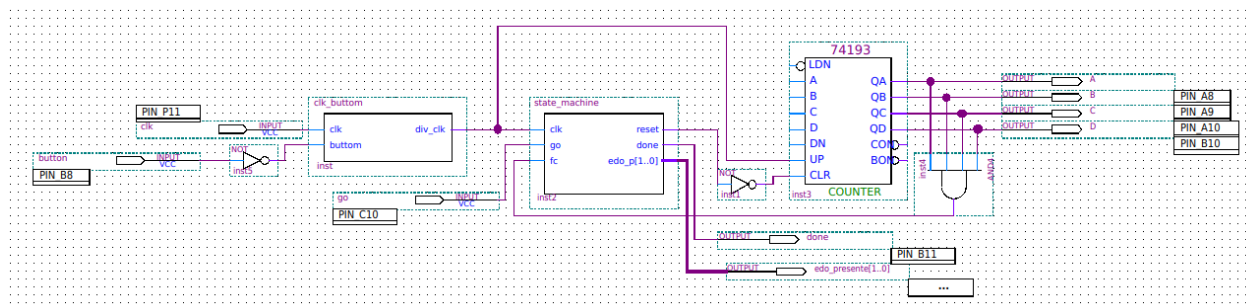


Figura 5: Diagrama de bloques completo del diseño implementado en Quartus.

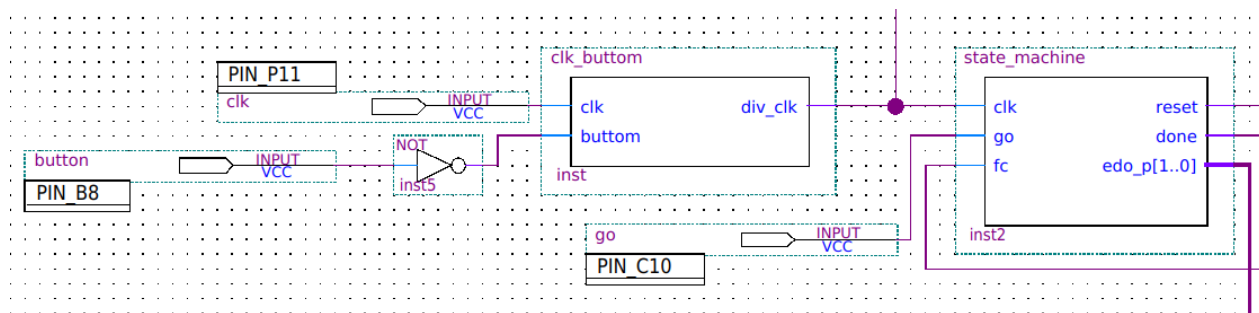


Figura 6: Detalle de la interfaz de reloj manual y de la máquina de estados.

En la Figura 6 se aprecia la ruta de control. El bloque `clk_button` filtra temporalmente la pulsación y entrega `div_clk`; de esta forma, una pulsación física produce sólo un avance lógico. La máquina principal recibe `go` y `fc`, y emite `reset`, `done` y el indicador `edo_p`.

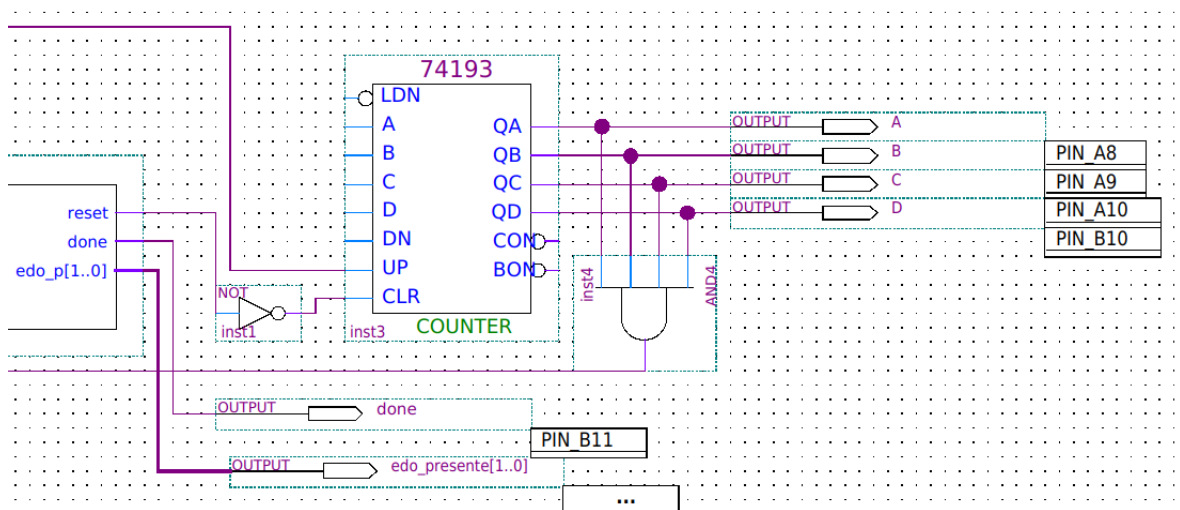


Figura 7: Detalle del contador, la detección de la cuenta final y las salidas de la FPGA.

En la Figura 7, la compuerta AND de cuatro entradas reconoce el valor 1111 del contador y genera `fc`. La salida `reset` se invierte para adecuarse a la entrada de borrado del contador. Con esta realimentación, el sistema puede reiniciar el contador al comenzar una nueva secuencia y emitir `done` al finalizarla.

4.1. Evidencia en tarjeta FPGA

Las fotografías de la implementación física se agregarán cuando se cuente con ellas. Se reservan los espacios mostrados en la Figura 8 para documentar la conexión de la tarjeta, el estado inicial del contador y una secuencia de avance al presionar el botón.

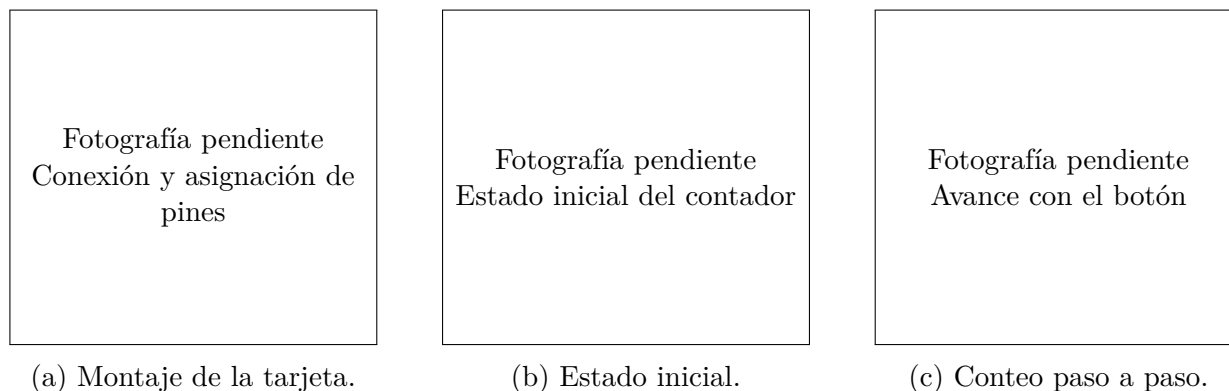


Figura 8: Espacios reservados para la evidencia experimental en la FPGA.

5. Conclusión

Percival Ulises Espinoza Matamoros: La práctica permitió trasladar una solución de lógica secuencial desde un diagrama de bloques a una descripción VHDL estructurada en módulos. El uso de estados con nombres explícitos facilitó relacionar la carta ASM con las transiciones implementadas en el código. Además, el control manual hizo más clara la observación de cada etapa del contador.

Oscar Iván Montiel Juárez: La generación de `div_clk` mediante el botón mostró la importancia de acondicionar las entradas físicas antes de usarlas como señales de control. El conteo interno de 25 millones de ciclos evita que una pulsación o sus rebotes provoquen varios avances. Las formas de onda permitieron verificar el estado presente, el conteo binario y el pulso de `done` antes de programar la FPGA.

Amy Valentina Veraza García: Se comprobó que VHDL permite describir y reutilizar de manera ordenada tanto una máquina de estados como un generador de pulsos. La integración con el contador y la detección de la cuenta máxima produjo un sistema funcional en el que cada pulsación controla el avance. La evidencia física pendiente complementará la validación obtenida en simulación.